

C Programming

Formulas & Operators: Ch.5

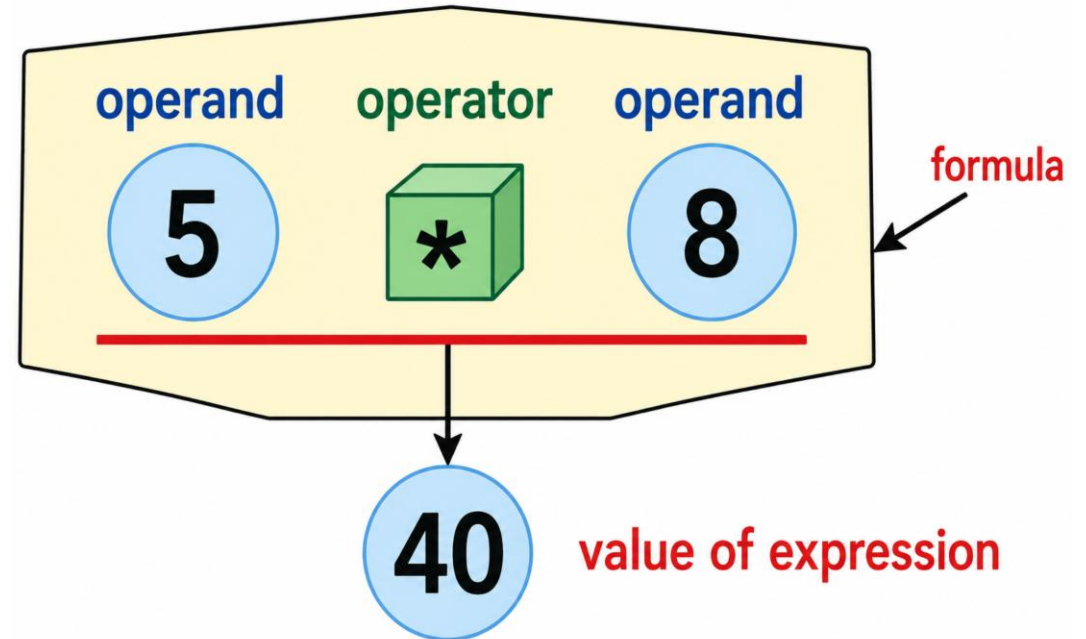
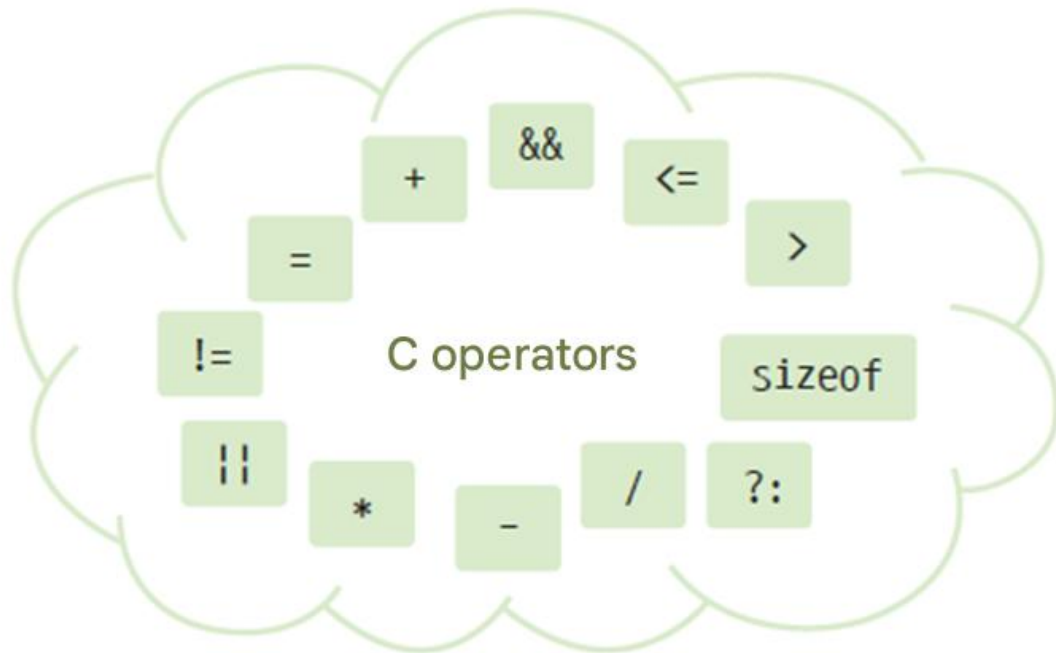
Operators | if / switch | while, do-while, for

Ch. 5

Formulas and Operators

A Computer Is Fundamentally a Calculating Machine

- Operators in the C language: the de facto industry standard - modern languages such as Java, C++, Python, and JavaScript use C language operators almost as they are.
- Formula (**expression**): constants, variables, and operators. It is divided into operators and operands.



Classification of operators by function

	Operators	Type
Unary Operator →	++, --	Unary Operator
Binary Operator {	+, -, *, /, %	Arithmetic Operator
	<, <=, >, >=, ==, !=	Relational Operator
	&&, , !	Logical Operator
	&, , <<, >>, ~, ^	Bitwise Operator
	=, +=, -=, *=, /=, %=	Assignment Operator
Ternary Operator →	?:	Ternary or Conditional Operator

Arithmetic Operators

- **Arithmetic Operations** : The Most Basic Operations on a Computer.
- Operators that perform basic arithmetic operations such as addition , subtraction , multiplication , and division.

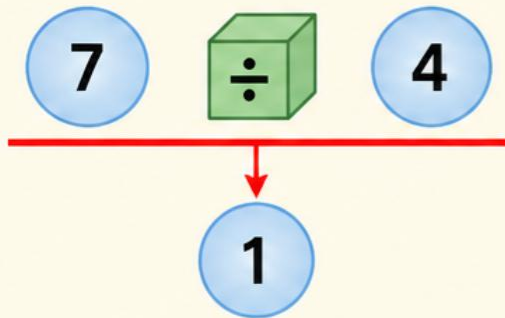
Operator	Sign	Example of Use	Result
Addition	+	$7 + 4$	11
Subtraction	-	$7 - 4$	3
Multiplication	*	$7 * 4$	28
Division	/	$7 / 4$	1
Remainder (Modulus)	%	$7 \% 4$	3

Next slide

Arithmetic Operators: Division and the Remainder

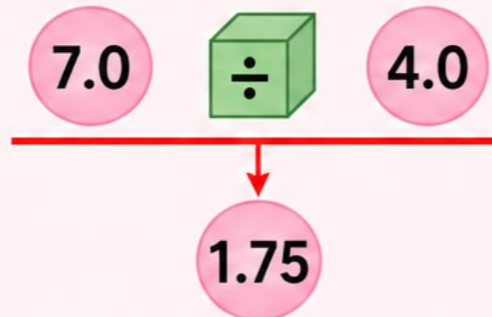
- Division between integers produces an integer result, and division between floating-point numbers produces a floating-point value. In integer division, the decimal places are discarded.
- The modulus operator % calculates the remainder when the first operand is divided by the second: $10 \% 2$ is 0, $5 \% 7$ is 5, $30 \% 9$ is 3.
- Example use: distinguishing even and odd numbers ($x \% 2 == 0$), or determining multiples of 3 ($x \% 3 == 0$).

Division of integers



$$7 \div 4 = 1 \text{ (remainder 3)}$$

Division of real numbers and real numbers



$$7.0 \div 4.0 = 1.75$$

Remainder (Modulus) Examples

The % operator returns the remainder after division.

$$10 \% 2 = 0$$

$$5 \% 7 = 5$$

$$30 \% 9 = 3$$

Example: Integer Arithmetic Operations

```
#include <stdio.h>

int main(void)
{
    int x, y, result;
    printf( "Enter two integers: ");
    scanf( "%d %d" , &x, &y);

    result = x + y;    printf( "%d + %d = %d" , x, y, result); // addition
    result = x - y;    printf( "%d - %d = %d" , x, y, result); // subtraction
    result = x * y;    printf( "%d * %d = %d" , x, y, result); // multiplication
    result = x / y;    printf( "%d / %d = %d" , x, y, result); // division
    result = x % y;    printf( "%d %% %d = %d" , x, y, result); // remainder
    return 0;
}
```

*Note: C has no exponentiation operator - simply multiply the variable by itself, like $x * x$.*

Example: Remainder Operator: Seconds to Minutes

```
// Remainder operator program
#include <stdio.h>
#define SEC_PER_MINUTE 60 // 1 minute is 60 seconds

int main( void )
{
    int input, minute, second;

    printf ( "Please enter seconds: " );
    scanf ( "%d" , &input);

    minute = input / SEC_PER_MINUTE; // how many minutes
    second = input % SEC_PER_MINUTE; // how many seconds

    printf ( "%d seconds is %d minutes %d seconds.\n" , input, minute, second);
    return 0;
}
```



Sign Operator

- Change the sign of a variable or constant.

```
x = -10;  
y = -x; // The value of variable y becomes 10.
```

Increment / Decrement Operators

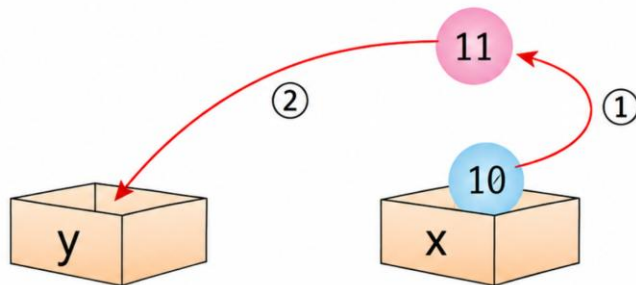
- ++, --: operators that **increase** or **decrease** the value of a variable by one.

- The difference between ++x and x++:

With ++x, the value is increased first and the increased value is used in the formula.

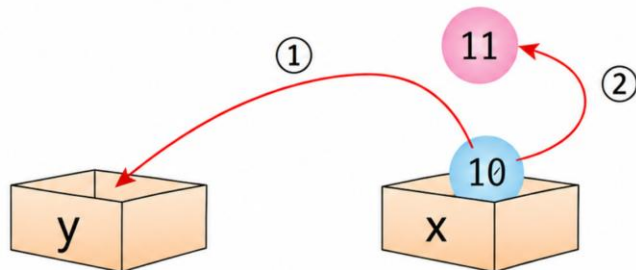
With x++, the current value is used in the formula first, and it increases later.

`y=++X;`



The increased value of x is assigned to y.

`y=X++;`



Substitute first, then increase later.

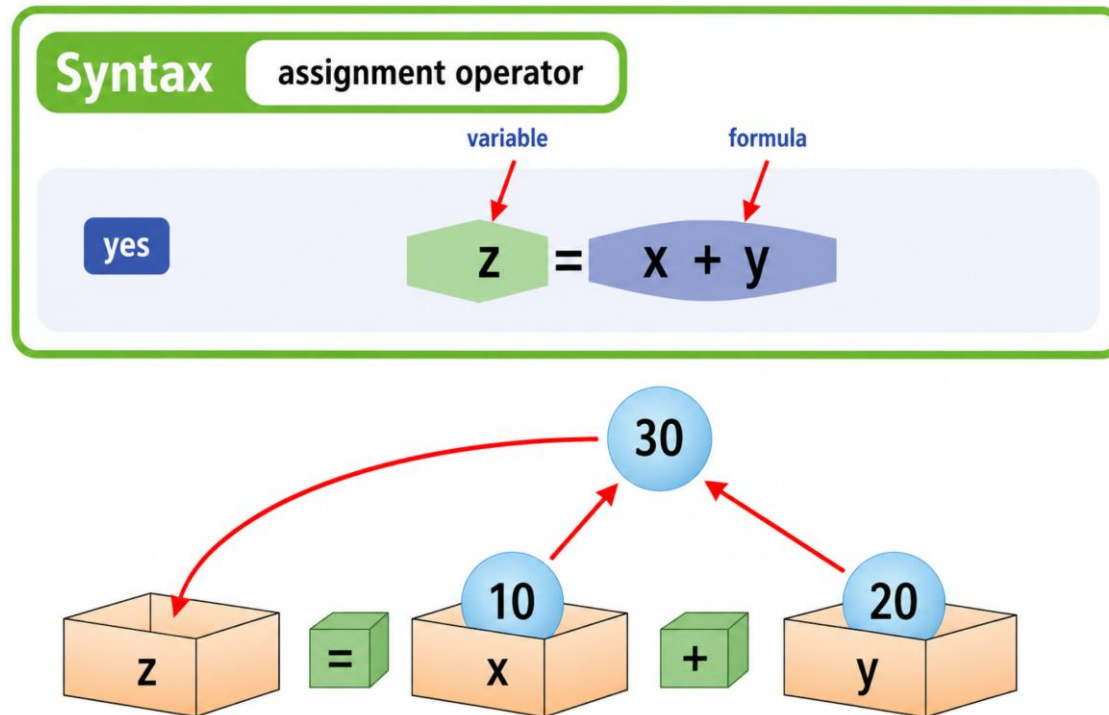
```
int x=10, y=10;
```

```
printf ( "x = %d\n", x);  
printf ( "++x value =% d\n", ++x);    // x = 11  
printf ( "x = %d\n\n", x);            // x = 11
```

```
printf ( "y = %d\n", y);  
printf ( "value of y++ = %d\n", y++);  // still 10  
printf ( "y = %d\n", y);               // now 11
```

Assignment Operator

- $100 = x + y;$ is a compile error : the left side of $=$ must be a variable, not a value.
- $x = x + 1;$ is a correct statement in C, but mathematically incorrect (in math, x can never equal $x+1$)
- All operations, including the assignment operation, have a result value.
- $y = x = 3;$ is possible: $x = 3$ is performed first, and the resulting value 3 is then assigned to y .



Compound Assignment Operator

- A compound assignment operator is an operator that combines

compound assignment operator	meaning	compound assignment operator	meaning
$x += y$	$x = x + y$	$x \&= y$	$x = x \& y$
$x -= y$	$x = x - y$	$x = y$	$x = x y$
$x *= y$	$x = x * y$	$x \wedge= y$	$x = x \wedge y$
$x /= y$	$x = x / y$	$x \gg= y$	$x = x \gg y$
$x \%= y$	$x = x \% y$	$x \ll= y$	$x = x \ll y$

$x *= y + 1$

$x \%= x + y$



$x = x * (y + 1)$

$x = x \% (x + y)$

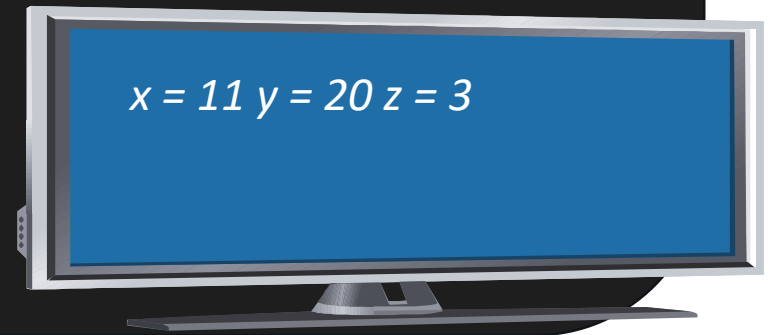
Compound Assignment Operator

```
// Compound assignment operator program
#include < stdio.h >

int main( void )
{
    int x = 10, y = 10, z = 33;

    x += 1;      // same as x = x + 1
    y *= 2;      // same as y = y * 2
    z %= 10 + 20; // same as z = z % (10 + 20)

    printf ( "x = %d y = %d z = %d \n" , x, y, z);
    return 0;
}
```



A compound assignment operator combines an operation and an assignment: $x += y$ has the same meaning as $x = x + y$

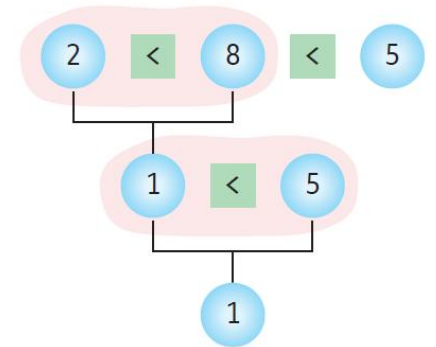
```
++x = 10;
```

```
x + 1 = 20;
```

```
x = *y;
```

Relational Operators

- An **operator** that compares two operands. The result is **true (1)** or **false (0)**.
- $x == y$ compares whether the values of x and y are equal.
- **Caution!** $(x = y)$ substitutes the value of y into x ; $(x == y)$ checks whether x and y are equal. Be careful not to use them incorrectly!
- **Caution!:** as in mathematics, writing $2 < x < 5$ together gives wrong results. The right way is $(2 < x) \ \&\& \ (x < 5)$.



operator	meaning	operator	meaning
$x == y$	Is x and y equal?	$x < y$	Is x less than y ?
$x != y$	Is x and y different?	$x >= y$	Is x greater than or equal to y ?
$x > y$	Is x greater than y ?	$x <= y$	Is x less than or equal to y ?

Examples: Relational operators

```
1 == 1 // true (1)
1 != 2 // true (1)
2 > 1  // true (1)
x >= y // true if x is greater than or equal to y (1) , otherwise false (0)
```

```
#include <stdio.h>
int main(void)
{
    int x, y;

    printf(" Enter two integers : ");
    scanf("%d%d", &x, &y);

    printf(" The result of x == y : %d", x == y);
    printf(" The result of x != y : %d", x != y);
    printf(" Result of x > y : %d", x > y);
    printf(" The result of x < y : %d", x < y);
    printf(" The result of x >= y : %d", x >= y);
    printf(" The result of x <= y : %d", x <= y);

    return 0;
}
```

```
Enter two integers : 3 4
The result of x == y : 0
The result of x != y : 1
Result of x > y : 0
The result of x < y : 1
The result of x >= y : 0
The result of x <= y : 1
```


Logical Operators

- An **operator** that combines multiple conditions to determine true or false. The result is true (1) or false (0).
- $x \ \&\& \ y$ (AND): **true** only if both x and y are **true**.
- $x \ || \ y$ (OR): **true** if at least one of x or y is **true**.
- $!x$ (NOT): if the operand is **true**, the result is **false**, and if the operand is **false**, the result is **true**.

Operation	Meaning
$x \ \&\& \ y$	AND operation: true only if both x and y are true; otherwise false.
$x \ \ y$	OR operation: true if at least one of x or y is true; false only if both x and y are false.
$!x$	NOT operation: reverses the value of x ; true if x is false, and false if x is true.

Logical Operators (&& AND)

AND (&&) Example

A company is hiring new employees and they set a requirement that the applicants be under 30 years old and have a TOEIC score of 700 or higher.

$(\overset{27}{\text{age}} < 30) \ \&\& \ (\overset{800}{\text{toeic}} \geq 700)$

$\underbrace{\hspace{10em}}_{\text{True(1)}}$

Both conditions are true (1), so the result of the AND (&&) operation is true (1).

Example

- $\text{age} = 27 \rightarrow 27 < 30 \rightarrow \text{True}(1)$
- $\text{toeic} = 800 \rightarrow 800 \geq 700 \rightarrow \text{True}(1)$
- $\text{True} \ \&\& \ \text{True} \rightarrow \text{True}(1)$

Logical Operators (|| OR)

OR (||) Example

The conditions for hiring new employees have changed so that they must be under 30 years old or have a TOEIC score of 700 or higher.

$(\overset{27}{\text{age}} < 30) \ || \ (\overset{500}{\text{toeic}} \geq 700)$

$\underbrace{\hspace{10em}}_{\text{True}(1)}$

The diagram shows the logical expression $(\text{age} < 30) \ || \ (\text{toeic} \geq 700)$. Above the word 'age' is the value 27, and above 'toeic' is 500. The word 'age' is circled in red, and 'toeic' is also circled in red. Below the first part of the expression, $(\text{age} < 30)$, is a bracket labeled 'True(1)'. Below the second part, $(\text{toeic} \geq 700)$, is a bracket labeled 'False(0)'. A larger bracket underneath both parts is labeled 'True(1)', indicating the final result of the OR operation.

At least one of the conditions is true (1), so the result of the OR (||) operation is true (1).

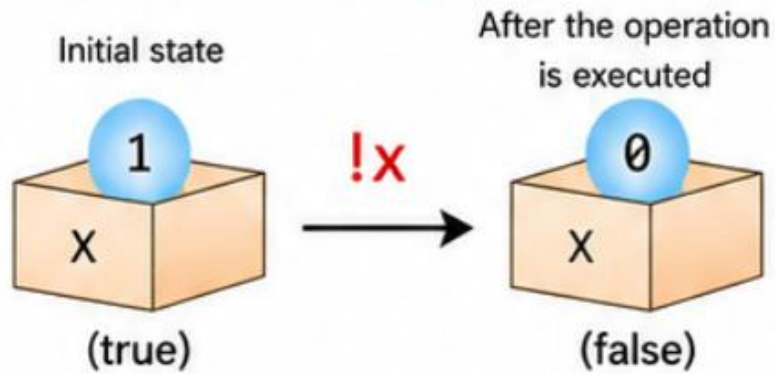
Example

- $\text{age} = 27 \rightarrow 27 < 30 \rightarrow \text{True}(1)$
- $\text{toeic} = 500 \rightarrow 500 \geq 700 \rightarrow \text{False}(0)$
- $\text{True} \ || \ \text{False} \rightarrow \text{True}(1)$

Logical Operators (! NOT)

NOT (!) Explanation

The NOT (!) operator reverses the value of its operand.



```
int result;  
result = !1;           // 0 is assigned to result  
result = !(2 == 3);    // 1 is assigned to result  
result = !(2 == 3);    // 1 is assigned to result
```

Examples

- `!1` → `!true` → `false(0)`
- `2 == 3` → `false(0)`
- `!(2 == 3)` → `!false` → `true(1)`

💡 In C, 0 is considered false, and any non-zero value is considered true.

```
!0 // The value of the expression is 1  
!3 // The value of the expression is 0  
!-3 // The value of the expression is 0
```

Examples: Logical Operators

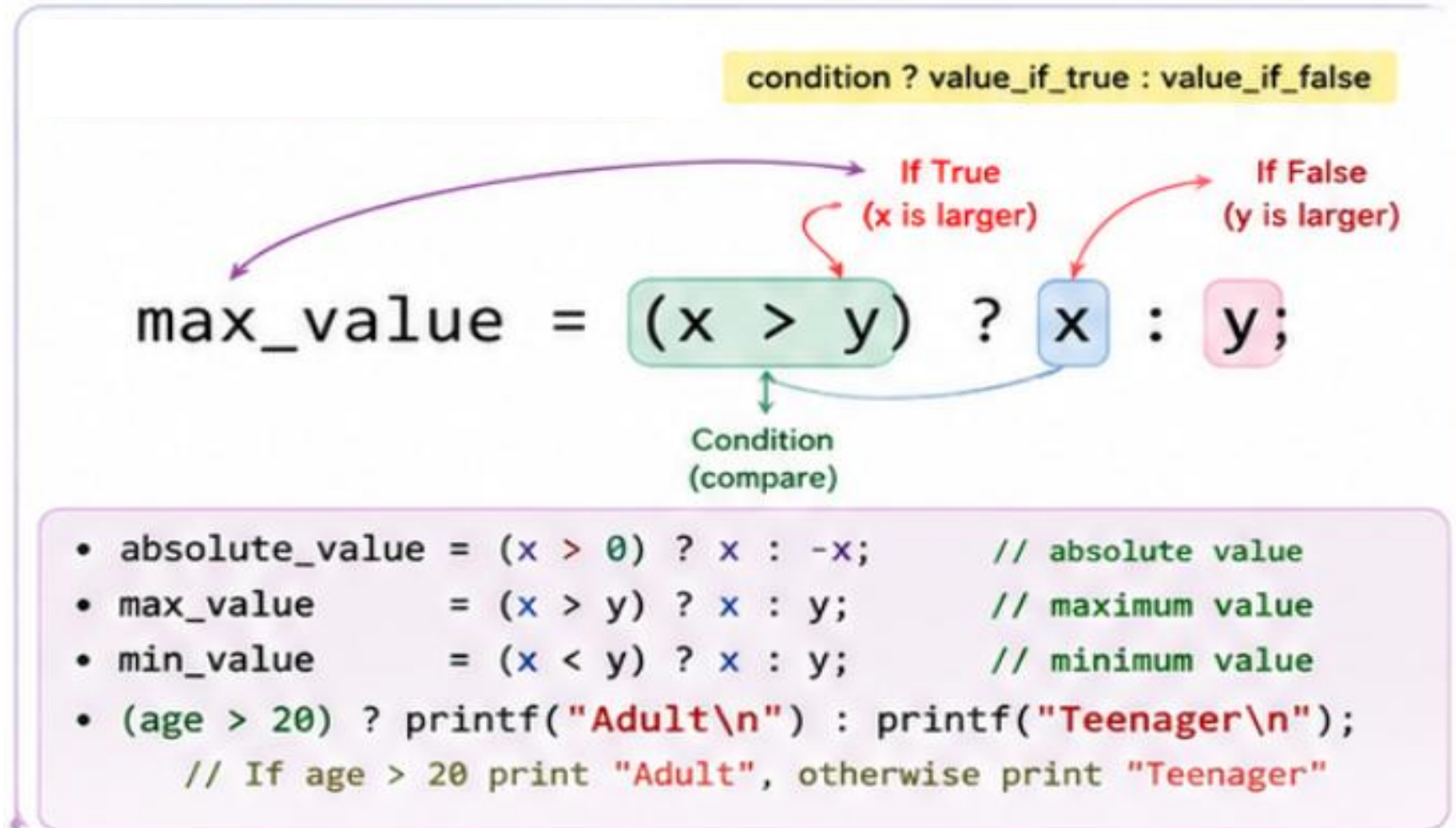
- “Is x one of 1, 2, or 3?”
`(x == 1) || (x == 2) || (x == 3)`
- “x is greater than or equal to 60 and less than 100.”
`(x >= 60) && (x < 100)`
- “x is neither 0 nor 1.”
`(x != 0) && (x != 1)`

Caution: Shortcut Evaluation

- For the **&& operator**, if the first operand is false, the other operands are not evaluated:
`(2 > 3) && (++x < 5)`
- For the **|| operator**, if the first operand is true, the other operands are not evaluated:
`(3 > 2) || (--x < 5)`
- Be careful - ++ and -- may not run if shortcut evaluation skips that part of the **expression**!

Ternary Operator (?:)

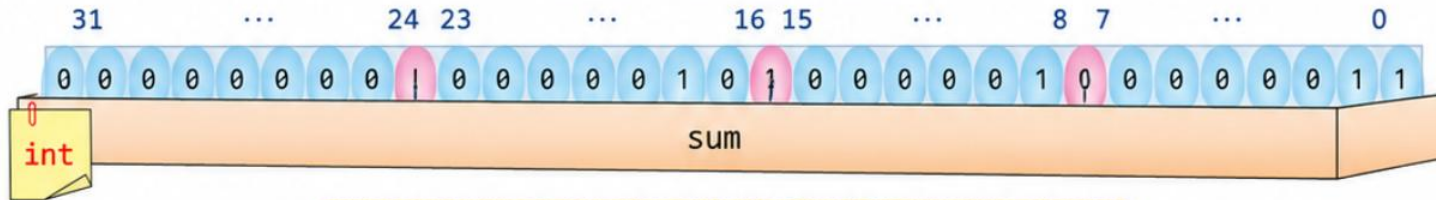
- A simple if-else statement can also be expressed using the ternary operator.



Bitwise Operators

1. int in C

int variables are 32 bits.



In C, int is commonly 32 bits (4 bytes), which can represent values from -2,147,483,648 to 2,147,483,647.

2. Bitwise Operators

Operator	Meaning	Description	Example (A = 12, B = 5)	Binary Example (A = 1100, B = 0101)
&	Bitwise AND	Result bit is 1 only if both corresponding bits are 1, otherwise 0.	A & B = 12 & 5 = 4	$\begin{array}{r} 1100 \text{ (12)} \\ \& 0101 \text{ (5)} \\ \hline 0100 \text{ (4)} \end{array}$
	Bitwise OR	Result bit is 1 if at least one of the corresponding bits is 1, otherwise 0.	A B = 12 5 = 13	$\begin{array}{r} 1100 \text{ (12)} \\ 0101 \text{ (5)} \\ \hline 1101 \text{ (13)} \end{array}$
^	Bitwise XOR	Result bit is 1 if the corresponding bits are different, otherwise 0.	A ^ B = 12 ^ 5 = 9	$\begin{array}{r} 1100 \text{ (12)} \\ ^ 0101 \text{ (5)} \\ \hline 1001 \text{ (9)} \end{array}$
<<	Left Shift	Shifts all bits to the left by a specified number. (Zero fill on the right)	A << 1 = 12 << 1 = 24	$\begin{array}{r} 1100 \text{ (12)} \\ << 1 \\ \hline 11000 \text{ (24)} \end{array}$
>>	Right Shift	Shifts all bits to the right by a specified number. (Zero fill on the left for unsigned)	A >> 1 = 12 >> 1 = 6	$\begin{array}{r} 1100 \text{ (12)} \\ >> 1 \\ \hline 0110 \text{ (6)} \end{array}$
~	Bitwise NOT	Inverts all bits. 0 becomes 1 and 1 becomes 0.	~A = ~12 = -13	$\begin{array}{r} \sim 000\dots1100 \text{ (12)} \\ \hline 111\dots0011 \text{ (-13)} \end{array}$

Bitwise Operators: Bit Shift

In C, int variables are 32 bits.

1. Left Shift (<<)

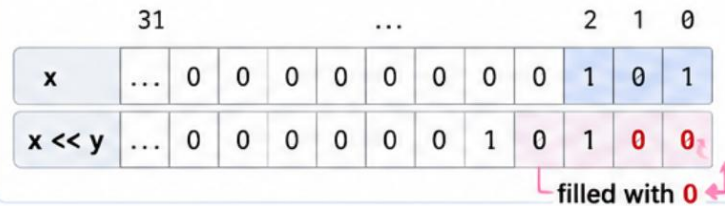
$x \ll y$ shifts the bits of x y places to the left.

- Vacated bits on the right are filled with 0.
- Same as multiplying by 2^y .

Example

$x = 5$ (binary: 000...00000101)

$y = 2$



Result

$x \ll 2 = 20$

(binary: 000...00010100)

(decimal: 20)

2. Right Shift (>>)

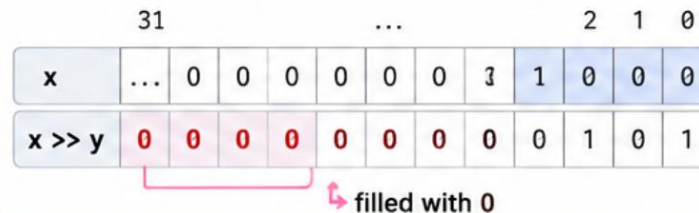
$x \gg y$ shifts the bits of x y places to the right.

- Vacated bits on the left are filled with 0.
- Same as dividing by 2^y (discards remainder).

Example

$x = 20$ (binary: 000...00010100)

$y = 2$



Result

$x \gg 2 = 5$

(binary: 000...00000101)

(decimal: 5)

Operator	Sign	Explanation
Shift Left	<<	$x \ll y$ shifts the bits of x y spaces to the left
Shift Right	>>	$x \gg y$ shifts the bits of x y places to the right

Quick Examples

- $5 \ll 1 = 10$ (000...0101 << 1 = 000...1010)
- $5 \ll 2 = 20$ (000...0101 << 2 = 000...10100)
- $20 \gg 1 = 10$ (000...10100 >> 1 = 000...01010)
- $20 \gg 2 = 5$ (000...10100 >> 2 = 000...00101)



Key Point: Left shift (<<) multiplies by 2^y (add zeros on the right).

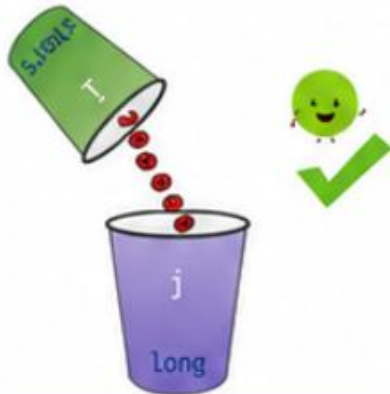
Right shift (>>) divides by 2^y (discard remainder, add zeros on the left).

Type Conversion

- Type conversion is changing the type of data during execution.

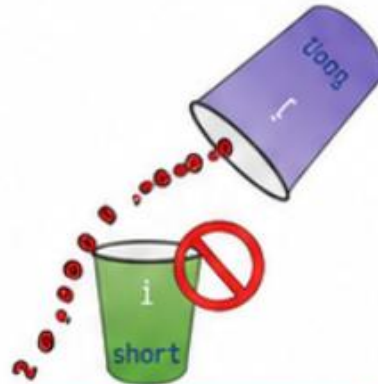
1. Why is it important?

✓ Correct conversion



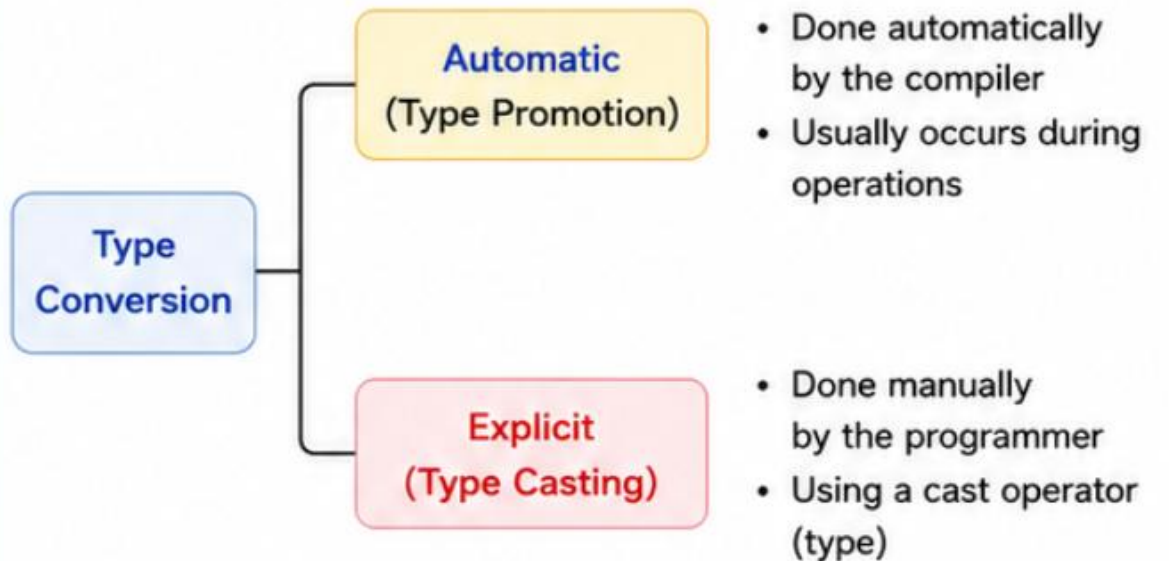
Data is safely preserved.

✗ Incorrect conversion



Data may be lost or corrupted.

2. Types of conversion



Automatic Type Conversion

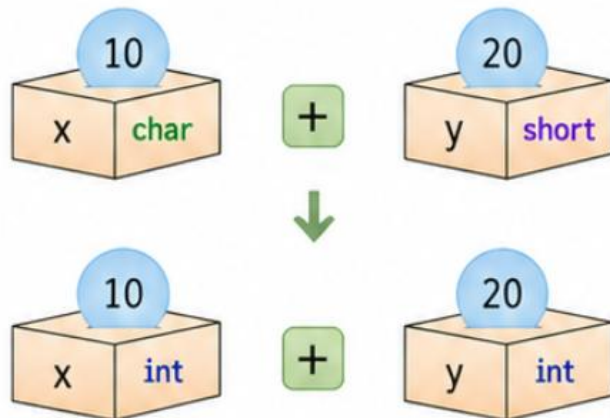
The compiler **automatically** changes data types during operations.



Purpose: To prevent data loss and perform operations correctly.

1. Integer Operations (char, short → int)

When performing integer operations, **char** or **short** is automatically converted to **int** type before calculation.

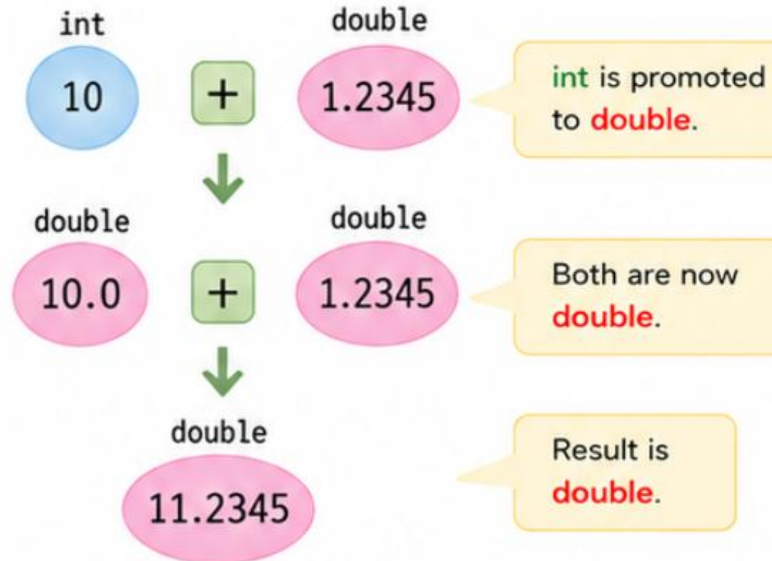


Operation performed in **int** type

$10 + 20 = 30$ (**int**)

2. Operations with Different Types (Promoted to Larger Type)

When different data types are used together, they are unified to the **larger data type**.



Operation performed in **double** type

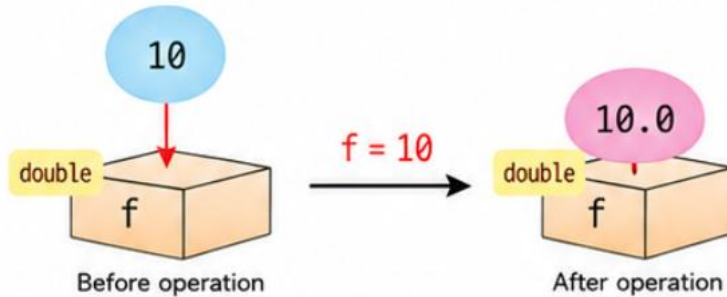
Automatic Type Conversion

3. Automatic type conversion during assignment operations

(1) Upward conversion (small → large)

- Smaller type is automatically converted to a larger type.
- No data is lost.

```
double f;  
f = 10; // 10.0 is stored in f
```



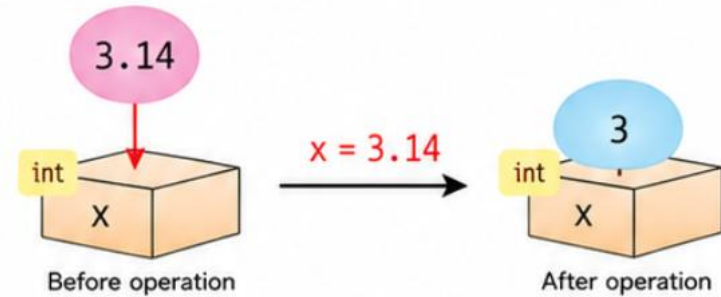
Examples

- int → double
- float → double
- int → long
- char → int

(2) Downward conversion (large → small)

- Larger type is automatically converted to a smaller type.
- Some data may be lost (precision loss).

```
int i;  
i = 3.141592; // 3 is stored in i
```



Examples

- double → int
 - float → int
- (Decimal part may be removed)



Key Points

- Upward conversion: safe, no data loss.
- Downward conversion: may cause data loss.
- Be careful when converting types!

Common Types (from small to large)

char → short → int → long → float → double

Smaller type values can be stored in larger types.
(Larger type values may not fit in smaller types.)

Explicit Type Conversion

Type conversion is **changing the type of data** during execution.



Explicit type conversion is done **manually** by the programmer using a cast operator (**type**).

1. SYNTAX

Cast Operator: (**type**)

YES

data type (to convert to)

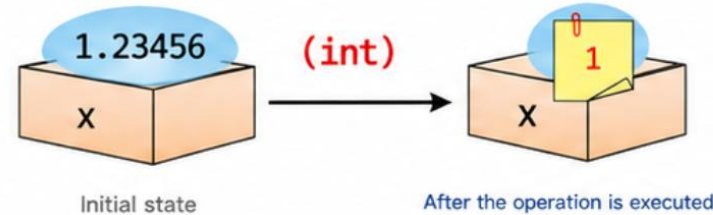
value or expression

resulting type

```
(int) 1.23456 → int // Convert to int type
(double) x → double // Convert to double type
(long) (x+y) → long // Convert to long type
```

2. HOW IT WORKS

The value or expression on the right is converted to the type specified in (**type**).



Example

```
double x = 1.23456;
x = (int) x; // x becomes 1
// Decimal part is removed.
```

3. COMMON EXPLICIT CONVERSIONS

To Type	Example	Result
int	(int) 3.14	3
double	(double) 10	10.0
long	(long) 100	100L
char	(char) 65	'A'
int	(int) 'A'	65



**Be careful
when converting!**

Decimal part may be
removed or value
may overflow.

4. NOTES

- When converting from a larger type to a smaller type, **data loss** may occur.
- Mainly used when the programmer wants a specific type.
- Improves code clarity and prevents unexpected results.

Example Program Output

```
1.000000
1.250000
1.250000
1.250000
3
2
```

Type Promotion Order
(from small to large)

char

<

short

<

int

<

long

<

float

<

double

Examples of Automatic Promotion

- int + int → int
- int + double → double
- double + float → double
- int + float → float
- (double) int / int → double
- (int) double → int (decimal part removed)

Examples: Type Conversion

In C, when data types are different in an operation, they are **automatically converted**.



Rule: In an expression, smaller types are promoted to a larger type. (**int** < **long** < **float** < **double**)

C Program

```
#include <stdio.h>

int main( void )
{
    int i;
    double f;

    // ----- Division operations -----
    f = 5 / 4;
    printf( "%f\n", f );

    f = ( double )5 / 4;
    printf( "%f\n", f );

    f = 5.0 / 4;
    printf( "%f\n", f );

    // ----- Addition operations (mixed types) -----
    f = ( double )5 / ( double )4;
    printf( "%f\n", f );

    i = 1.3 + 1.8;
    printf( "%d\n", i );

    i = ( int )1.3 + ( int )1.8;
    printf( "%d\n", i );

    return 0;
}
```

1. DIVISION OPERATIONS (Results stored in f (double))

Expression	What happens?	Result (stored in f)
5 / 4	5 and 4 are int . 5 / 4 = 1 (integer division) 1 is converted to 1.0 (double)	1.000000
(double) 5 / 4	5 is converted to double (5.0). 5.0 / 4 = 1.25	1.250000
5.0 / 4	5.0 is double . 5.0 / 4 = 1.25	1.250000

2. ADDITION OPERATIONS (Mixed Types)

Expression	What happens?	Result (stored in)
(double) 5 / (double) 4	5 and 4 are converted to double . 5.0 / 4.0 = 1.25	f (double) 1.250000
1.3 + 1.8	1.3 and 1.8 are double . 1.3 + 1.8 = 3.1 3.1 is converted to 3 (int)	i (int) 3
(int) 1.3 + (int) 1.8	1.3 → 1 (int), 1.8 → 1 (int) 1 + 1 = 2	i (int) 2

Program Output

```
1.000000
1.250000
1.250000
1.250000
3
2
```



Note: Integer division occurs only when both operands are **int**.

KEY POINTS

- ↑ Smaller types are **automatically converted** to larger types during operations.
- ⚡ If both operands are **int**, the result is **int** (integer division).
- 1.2 If one operand is **float** or **double**, both are promoted to that type.
- () You can use **casts** (**type**) to convert explicitly.

TYPE PROMOTION ORDER (from small to large)

char < **short** < **int** < **long** < **float** < **double**



When different types are used in an expression, the **smaller** type is promoted to the **larger** type in the above order.

EXAMPLES OF AUTOMATIC PROMOTION

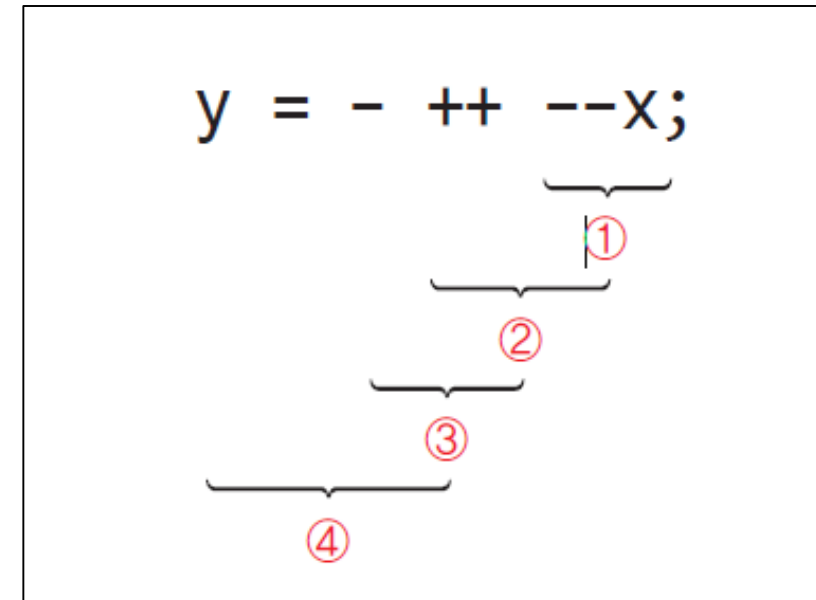
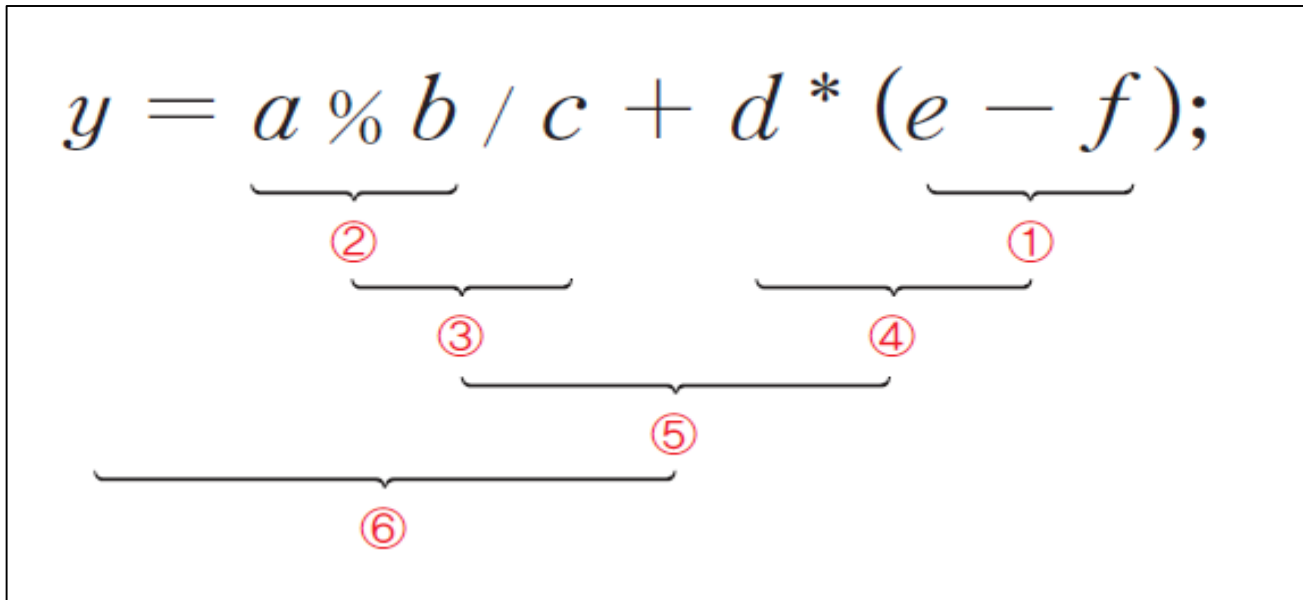
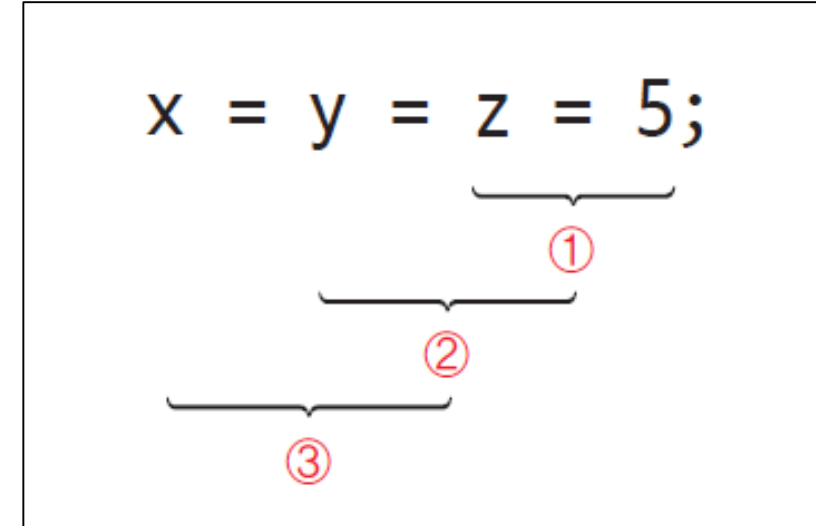
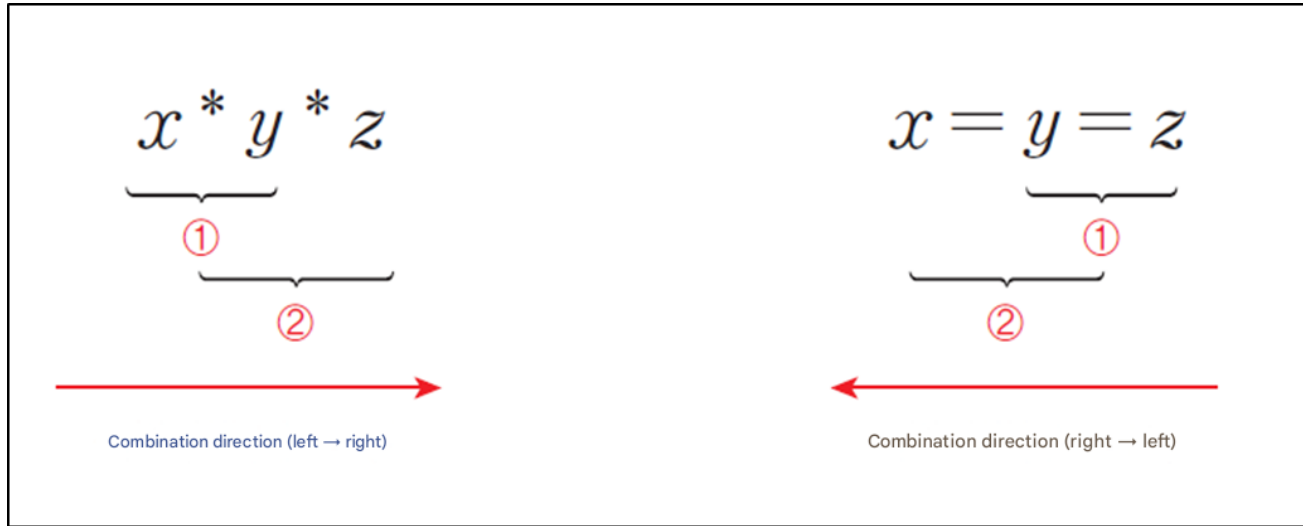
- **int** + **int** → **int**
- **int** + **double** → **double**
- **double** + **float** → **double**
- **int** + **float** → **float**
- (double) **int** / **int** → **double**
- (int) **double** → **int** (decimal part removed)

General Guidelines for Operator Priority

- Comma < Assignment < Logic < Relation < Arithmetic < Unary
- Parentheses have the highest precedence; all unary operators have higher precedence than binary operators.
- Assignment operators have the lowest precedence, except for the comma **operator**.
- Relational and logical operators have lower precedence than arithmetic operators: $x + 2 == y + 3$
- Relational operators have higher precedence than logical operators, so you can write: $x > y \ \&\& \ z > y$ (same as $(x > y) \ \&\& \ (z > y)$)
- If you can't remember the precedence of operators, **just use parentheses**.

Parentheses operators have the highest precedence.

General Guidelines for Operator Priority



Q & A

Lab & Quiz: right after this - see you in the practice session!